

DEFENSIVE AUTHENTICATION MONITORING & DETECTION ENGINEERING

SentinelLog

From raw authentication telemetry to actionable, structured security signals.

Python 3.11+ · Rule Engine · Audit Pipeline · Security Operations

Project Architecture & Detection Strategy · Authored by Debraj Dutta

SENTINELLOG ENGINE

AUTH-001 · AUTH-002 · AUTH-003

Authentication logs contain the signal — but not the story

Isolated log lines capture discrete authorization outcomes but obscure temporal patterns, distributed spray attempts, and session intent.

STAGE 01 · INGESTION

Raw Authentication Events

Fragmented timestamps, differing syslog schemas, and variable IP formats arrive from Linux hosts, IAM gateways, and internal services.

STAGE 02 · VOLUME

High-Noise Environments

Routine typos, batch jobs, and automated service retries produce tens of thousands of harmless single-line failures daily.

STAGE 03 · VISIBILITY

Missed Relationships

Cross-account spraying and slow brute force escape single-line grep commands and stateless metric alerts without stateful correlation.

STAGE 04 · CONSEQUENCE

Cumulative Security Risk

Legitimate credential stuffing and successful compromise after brute force go unnoticed until post-incident forensics.

Manual triage of stateless auth logs causes triage fatigue and extended dwell time.

Security analysts require context-enriched event streams rather than raw, isolated log records.

OPERATIONAL CHALLENGE

Grepping for `sshd Failed password` yields occurrences, not malicious intent. Analysts must manually assemble actor, temporal scope, target breadth, and final outcome.

SentinelLog turns telemetry into investigation-ready signals

CORE ARCHITECTURAL PROPOSITION

Deterministic parsing, stateful time-window correlation, and enriched triage payloads.

01

Normalize

Maps heterogeneous syslog, OpenSSH, and JSON schemas into a unified, type-safe `AuthEvent` model.

02

Correlate

Aggregates records within sliding temporal windows grouped by composite keys (IP, username, service).

03

Detect

Executes deterministic logic across event sequences to surface spraying, brute force, and anomalous follow-up success.

04

Emit

Generates self-contained `SecurityAlert` JSON payloads equipped with full forensic metadata and analyst guidance.

05

Extend

Modular Python engine enables engineers to add bespoke detection rules and parsers in under 50 lines of code.

Explicit boundary: SentinelLog produces investigation signals for defensive analysts, not automated blocks or unverified accusations of malicious compromise.

A compact pipeline from log line to alert

STAGE 1 · INGEST

Log Telemetry

- OpenSSH
/var/log/auth.log
- SentinelLog standard JSON
- Syslog daemon streams

Input Interface: File / STDIN

STAGE 2 · PARSE

Event Normalizer

- Regex & token extraction
- Timestamp canonicalization
- Yields typed AuthEvent

Artifact: Dataclass Model

STAGE 3 · EVALUATE

Detection Engine

- Sliding memory windows
- AUTH-001 (Single target)
- AUTH-002 (Spray pattern)
- AUTH-003 (Compromise clue)

Engine: Rule Registry

STAGE 4 · OUTPUT

Security Alert

- Severity & confidence tag
- Contextual event trace
- Terminal CLI / SIEM JSON

Target: SOC Triage Queue

EXECUTION MODEL

Stream-based processing ensures linear memory complexity $O(k)$ bounded by active sliding windows, preventing memory bloat during multi-gigabyte log rehydration.

Zero External DB: Self-contained Python runtime executes locally without heavyweight daemon dependencies.

Different inputs, one normalized event model

The normalizer resolves semantic discrepancies between freeform syslog strings and machine-native JSON into a rigid schema.

Unified Schema Field	OpenSSH syslog (auth.log)	Native SentinelLog JSON	Field Type & Normalization Rule
timestamp	Jan 15 10:00:00	2026-01-15T10:00:00Z	datetime (UTC ISO 8601)
event_type	Parsed from sshd[...] message	"AUTH_FAILURE"	Enum: FAILURE SUCCESS UNKNOWN
source_ip	Extracted via regex from from <ip>	"192.0.2.10"	IPv4 / IPv6 Address object
username	Extracted from for <user>/invalid user	"analyst"	str (Sanitized lowercased)
service	Process identifier: sshd	"ssh"	str (Normalized protocol/service)

Schema Stability: Downstream detection rules never touch regexes or raw line formats. All correlation logic interacts strictly with the typed AuthEvent interface.

EXTENSIBILITY HOOK

Adding support for Okta system log, AWS CloudTrail ConsoleLogin, or Windows Event ID 4625 requires solely writing a dedicated parser subclass yielding AuthEvent.

The engine detects patterns, not isolated lines

Stateless alerts scream at single failures; stateful detection establishes operational context across time and entity boundaries.

01

Event Stream

Incoming sequence of normalized events ordered chronologically.

02

Sliding Window

Ephemeral memory bucket bounded by duration (e.g. 300 seconds).

03

Correlation Key

Indexed by composite keys:
(source_ip) or (ip, user).

04

Rule Evaluator

Evaluates stateful thresholds, set cardinality, and sequence order.

05

Security Alert

Emits structured finding with aggregated context and severity.

TRADITIONAL LOG GREP (STATELESS)

- Fires on every failed login event.
- Drowns analysts in routine mistyped password noise.
- Blind to distributed timing and multi-user spraying.

SENTINELLOG CORRELATION (STATEFUL)

- Maintains time-series state per source and target entity.
- Suppresses isolated drops; alerts on concerted volume.
- Discovers compound behaviors across multiple accounts.

AUTH-001 · Repeated Authentication Failures

RULE LOGIC SPECIFICATION

Targeted Brute-Force Detection

Detects repeated authentication failures originating from a single source IP against a single username within a defined sliding time window.

Rule ID	AUTH-001
Severity	Medium (Investigation Signal)
Correlation Key	(source_ip, username)
Threshold	≥ 5 failed events within 300 seconds
Primary Threat	Credential brute forcing, dictionary attack, or broken automated service

ANALYST TRIAGE GUIDANCE

- 1. **Verify IP ownership:** Check if 192.0.2.10 belongs to internal VPN, proxy, or unknown external host.
- 2. **Account criticality:** Assess whether target user holds administrative or privileged tier access.
- 3. **Concurrency check:** Inspect whether other IPs are attempting the same user in parallel.

10:00:12 · FAIL attempt #1	10:01:04 · FAIL attempt #2	10:01:48 · FAIL attempt #3	10:02:30 · FAIL attempt #4	10:03:15 · FAIL ALERT TRIGGERED (5/5)
-------------------------------	-------------------------------	-------------------------------	-------------------------------	---

AUTH-002 · Cross-Account Failure Pattern

RULE LOGIC SPECIFICATION

Password Spraying Detection

Detects a single source IP probing multiple distinct usernames with authentication failures over a sliding temporal window, bypassing single-account lockout controls.

Rule ID	AUTH-002
Severity	Medium / High (Horizontal Probing)
Correlation Key	(source_ip)
Threshold	≥ 3 distinct usernames with failures in 600 seconds
Primary Threat	Horizontal password spray, automated user enumeration

SPRAY DEFENSE NUANCE

Why single-user rules fail: Password sprayers intentionally attempt only 1 or 2 passwords per user to avoid traditional lockout policies.

SentinelLog defense: By tracking unique username cardinality `len(set(users)) ≥ 3` per source IP, the attack is unmasked regardless of per-user pacing.

Source IP: 192.0.2.10

Target 1: admin

Failed password at 10:10:02

Source IP: 192.0.2.10

Target 2: root

Failed password at 10:11:45

Source IP: 192.0.2.10

Target 3: deploy

ALERT TRIGGERED (≥3 targets)

AUTH-003 · Success after repeated failures

RULE LOGIC SPECIFICATION

Compromise / Break-In Signal

Detects a successful authentication event preceded by a cluster of authentication failures for the same entity within a 900-second window.

Rule ID	AUTH-003
Severity	High / Critical Alert (Active Access Gained)
Correlation Key	(source_ip, username)
Condition	≥ 3 failures followed by 1 SUCCESS within 900s
Urgency	Potential account takeover, successful brute force, or valid password guess

IMMEDIATE RESPONSE VALUE

Differentiating Context: A success login by itself looks completely benign in audit logs.

The Forensic Clue: Tying that success directly to the preceding failure sequence transforms routine access into an urgent active session containment review.

10:20:10 · 192.0.2.10 FAILED (analyst)	10:21:05 · 192.0.2.10 FAILED (analyst)	10:22:30 · 192.0.2.10 FAILED (analyst)	10:23:14 · 192.0.2.10 SUCCESSFUL LOGIN · ALERT
---	---	---	---

Structured alerts keep the next step clear

Field	Value
alert_id	ALT-2026-0091
rule_id	AUTH-001
severity	medium
confidence	high
source_ip	192.0.2.10
username	analyst
failure_count	5
first_seen	2026-01-15T10:00:00Z
last_seen	2026-01-15T10:04:12Z
message	Repeated authentication failures for the same source and username
recommended_action	Verify host ownership for 192.0.2.10 and review active sessions.

Author: Debraj Dutta · Architecture: Python 3.11+ / Dataclasses / Unit Tested · Repository: <https://github.com/debrajdutta/SentinelLog>

Engineering & Validation

A modular, tested, and continuously validated Python implementation

Standard-Library-Focused

No heavyweight external dependencies. Pure Python 3.11+ dataclasses, regex, and collections.

Modular Architecture

Parser → Event → Detector → CLI. Each component independently testable and extensible.

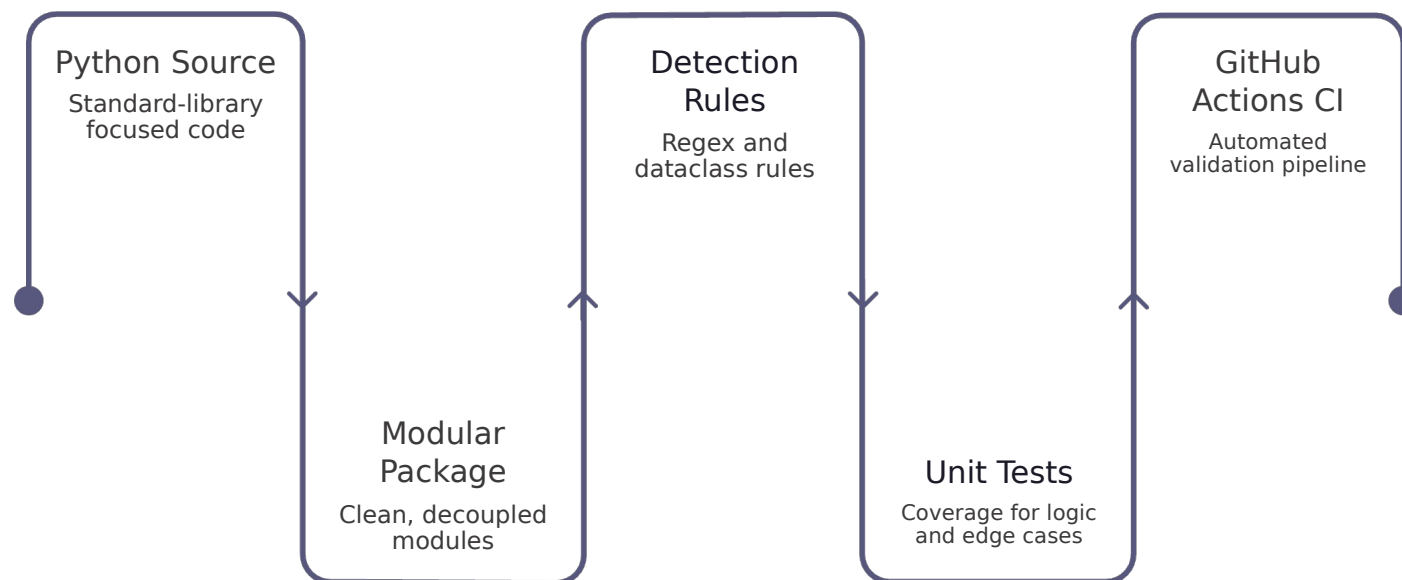
Deterministic Tests

Unit test suite validates parser correctness, event normalization, and detection rule logic in isolation.

Automated CI Validation

GitHub Actions pipeline tests across Python 3.11, 3.12, and 3.13 on every commit.

Validation Pipeline



From Telemetry to Alert

Synthetic demonstration: How raw events become investigation-ready signals



Example Alert — *Synthetic Data*

Field	Value
Source IP	192.0.2.10
Username	alice
Pattern	Failed × 5
Rule Triggered	AUTH-001
Severity	Medium

📄 *Synthetic demonstration data — no real unauthorized access depicted*

Security Boundaries & Limitations

Honest scope definition for defensive security operations

✓ Designed For

Authorized security monitoring

Local security labs

Detection engineering research

Authentication telemetry analysis

Security automation experiments

⚠ Current Boundaries

Alerts are investigation signals, not proof

Thresholds can produce false positives

Current scope: authentication telemetry only

Not a full SIEM replacement

Does not establish attacker intent by itself

Core Principle: SentinelLog produces investigation signals for defensive analysts. Alerts require human judgment and contextual verification

From Detection Engine to Security Monitoring Platform

Roadmap: Current capabilities and planned evolution

CURRENT

- Core Detection Engine
- CLI Interface
- AUTH-001 Rule Set
- Modular Parser

NEXT

- Stateful Real-Time Tailing
- Rule Configuration UI
- Alert Persistence
- Pluggable Output Sinks

FUTURE

- Security Metrics & Dashboards
- Additional Auth Formats
- Full SIEM Integration
- Multi-Source Correlation

 Future features are planned enhancements. Current release focuses on the core detection engine and CLI interface.

SentinelLog

Turning authentication telemetry into actionable security signals

DETECTION ENGINEERING

PYTHON

DEFENSIVE SECURITY

Telemetry

Collect raw authentication events from sources.

Parse

Normalize logs into structured fields for analysis.

Correlate

Link related events across time and entities.

Detect

Identify suspicious patterns and anomalies.